

TRANSFORMER-BASED SENTIMENT ANALYSIS

With Attention Visualization and Explainability (SHAP & LIME)

Submitted by:

Urwa Tehseen

Roll No: 23F-0008 | Section 6A

Natural Language Processing
May 2026

1. Introduction

Sentiment analysis is one of the most widely applied tasks in natural language processing (NLP), enabling systems to automatically determine whether a piece of text expresses a positive or negative opinion. This project implements a transformer-based binary sentiment classifier trained on the IMDb movie review dataset. The goal is not only to build an accurate classifier, but to understand how and why the model makes its predictions by integrating attention visualization and post-hoc explainability techniques.

The project fine-tunes a DistilBERT model on a balanced subset of the IMDb corpus, evaluates its performance on a held-out test set, and then applies three complementary interpretability methods: attention weight extraction, SHAP (SHapley Additive exPlanations), and LIME (Local Interpretable Model-Agnostic Explanations). Together, these techniques provide both quantitative performance evidence and qualitative insight into the model's decision-making behavior.

2. Methodology Flowchart

The pipeline follows a structured, end-to-end workflow from raw data ingestion through evaluation and explanation. Figure 1 presents the complete methodology as a flowchart.

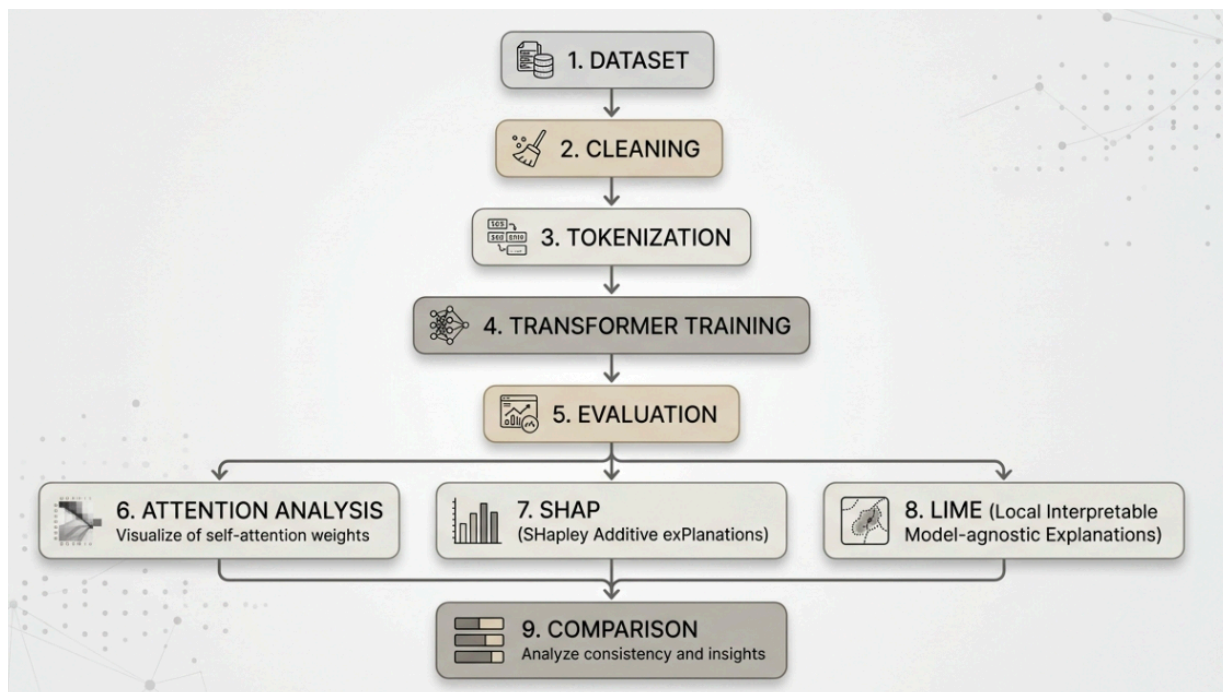


Figure 1: End-to-end methodology flowchart: data loading, preprocessing, model training, evaluation, attention visualization, and explainability.[made with gemini]

The pipeline consists of five principal stages: (1) data acquisition and cleaning, (2) tokenization and encoding using the DistilBERT tokenizer, (3) model fine-tuning with a classification head, (4) performance evaluation using standard metrics, and (5) post-hoc interpretability analysis using attention weights, SHAP, and LIME.

3. Dataset and Preprocessing

3.1 Dataset

The IMDb Large Movie Review Dataset was used for this experiment. It contains 50,000 movie reviews evenly split between positive and negative sentiments. For computational efficiency, a stratified subset was sampled for training and evaluation. The final test split consists of 1,000 samples, with 487 negative and 513 positive reviews, ensuring near-perfect class balance.

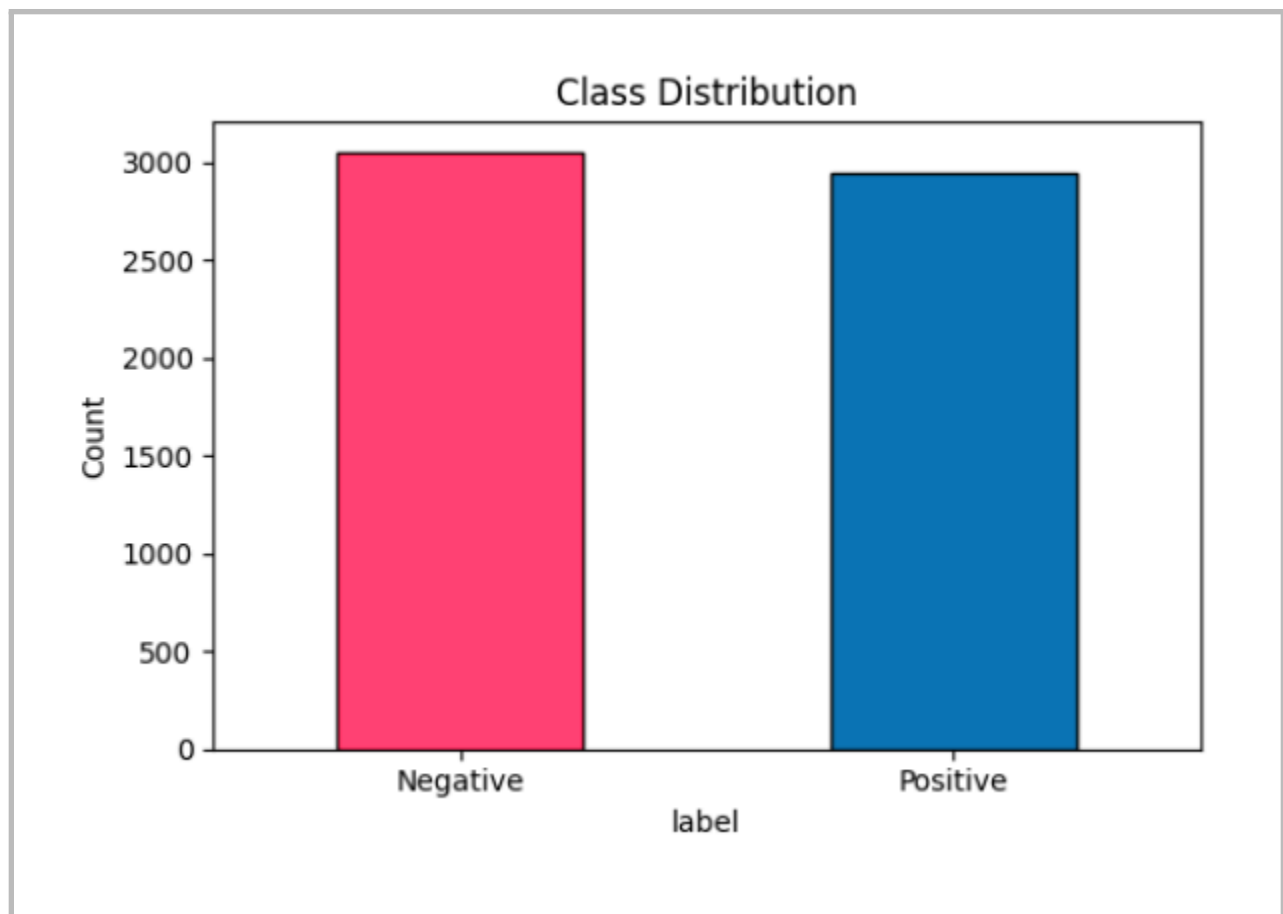


Figure 2: Class distribution (left) showing near-balanced negative and positive samples. Word count distribution (right) with a dashed line at max_len=128 tokens.

The class distribution plot confirms balanced sampling. The word count distribution shows that the majority of reviews fall between 20 and 150 words, with relatively few exceeding the chosen max sequence length of 128 tokens. Truncating at 128 tokens retains most of the semantic content while keeping computation tractable.

3.2 Preprocessing Steps

- HTML tags and special characters were removed from raw review text.
- Text was lowercased and stripped of leading and trailing whitespace.
- Tokenization was performed using the HuggingFace DistilBERT tokenizer with padding and truncation to a maximum length of 128 tokens.
- Input IDs and attention masks were generated and converted to PyTorch tensors.
- Data was loaded into PyTorch DataLoaders with shuffling enabled for training.

4. Transformer Model Architecture

The classifier is built on DistilBERT (Sanh et al., 2019), a distilled version of BERT that retains approximately 97% of BERT's language understanding capability at 40% fewer parameters. DistilBERT uses 6 transformer encoder layers, 12 attention heads per layer, and a hidden dimension of 768.

A linear classification head was appended to the [CLS] token representation produced by the final encoder layer. This [CLS] token aggregates contextual information from the entire input sequence and serves as the sentence-level embedding for binary classification. The model was fine-tuned end-to-end using the AdamW optimizer with a learning rate of $2e-5$ and a linear warmup schedule.

Dropout regularization with a rate of 0.1 was applied to prevent overfitting on the relatively small training subset.

5. Evaluation Metrics

Model performance was assessed on the 1,000-sample test set using standard binary classification metrics. Table 1 summarizes the per-class and aggregate results.

Table 1: Classification report on the test set (n=1000)

Class	Precision	Recall	F1-Score	Support
Negative	0.93	0.92	0.92	487
Positive	0.93	0.93	0.93	513
Macro Avg	0.93	0.93	0.93	1000
Weighted Avg	0.93	0.93	0.93	1000

The model achieves an overall accuracy of 93%, with balanced precision and recall across both classes. The macro-averaged F1-score of 0.93 confirms that the model does not favor one class over the other, which is expected given the balanced dataset.

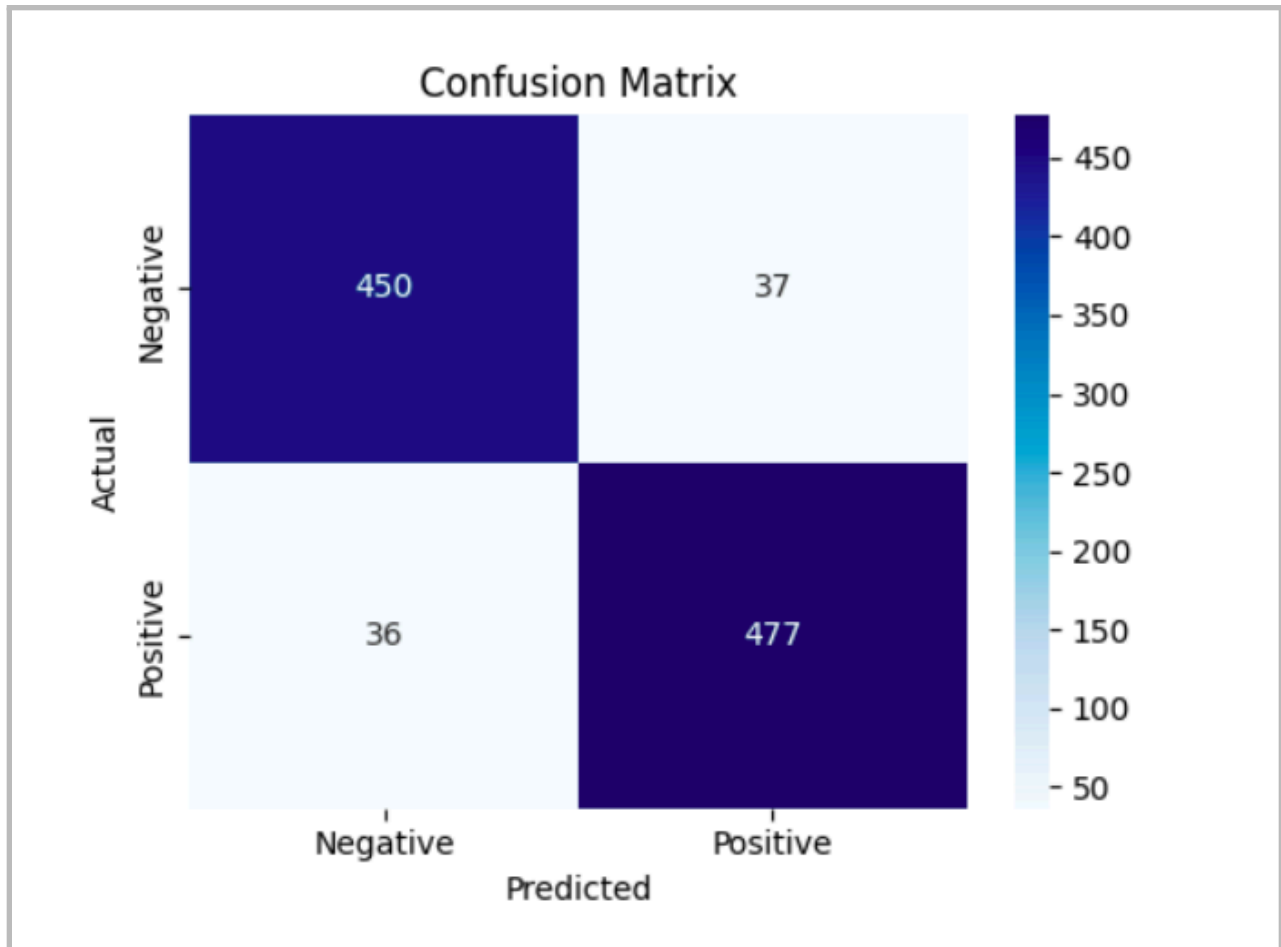


Figure 3: Confusion matrix showing 450 true negatives, 477 true positives, 37 false positives, and 36 false negatives out of 1,000 test samples.

The confusion matrix shows that misclassifications are symmetric: 37 negative reviews were incorrectly labeled as positive (false positives), and 36 positive reviews were incorrectly labeled as negative (false negatives). This symmetry indicates the model is not systematically biased toward either sentiment class.

6. Attention Analysis

Transformer models compute multi-head self-attention at every layer, allowing each token to attend to every other token in the sequence. Visualizing these attention patterns provides insight into which words the model focuses on when forming its predictions.

6.1 Layer 1, Head 1: Positive Review

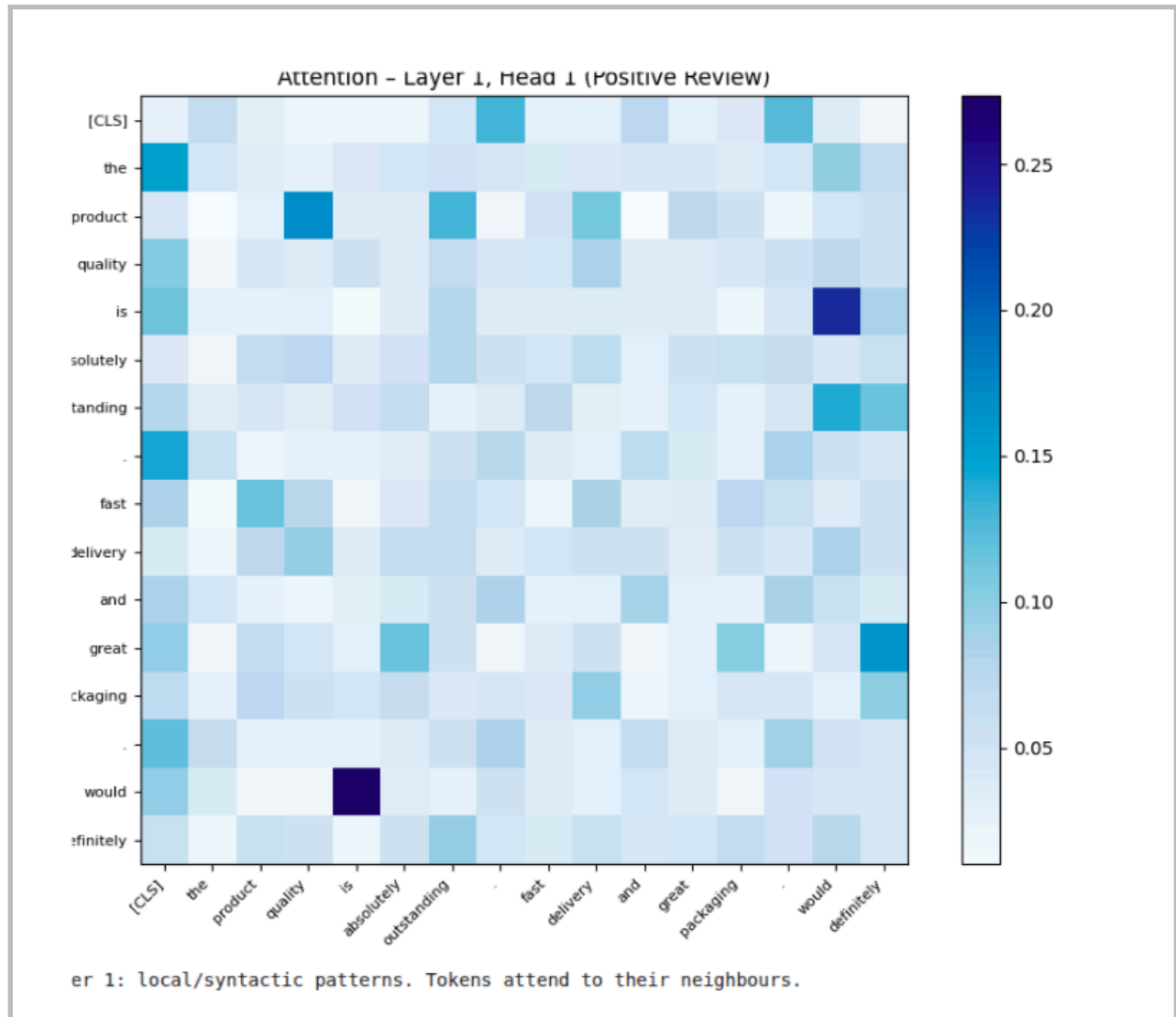


Figure 4: Attention heatmap from Layer 1, Head 1 on a positive review. Tokens attend primarily to their immediate neighbors, reflecting local syntactic pattern capture.

In the early layers, attention heads tend to capture local, syntactic relationships. The heatmap shows that each token distributes moderate attention to its adjacent tokens rather than forming strong long-range connections. This behavior is characteristic of the first transformer layer, which learns structural patterns such as subject-verb proximity rather than semantic sentiment signals.

6.2 Layer 6, Head 4: Negative Review

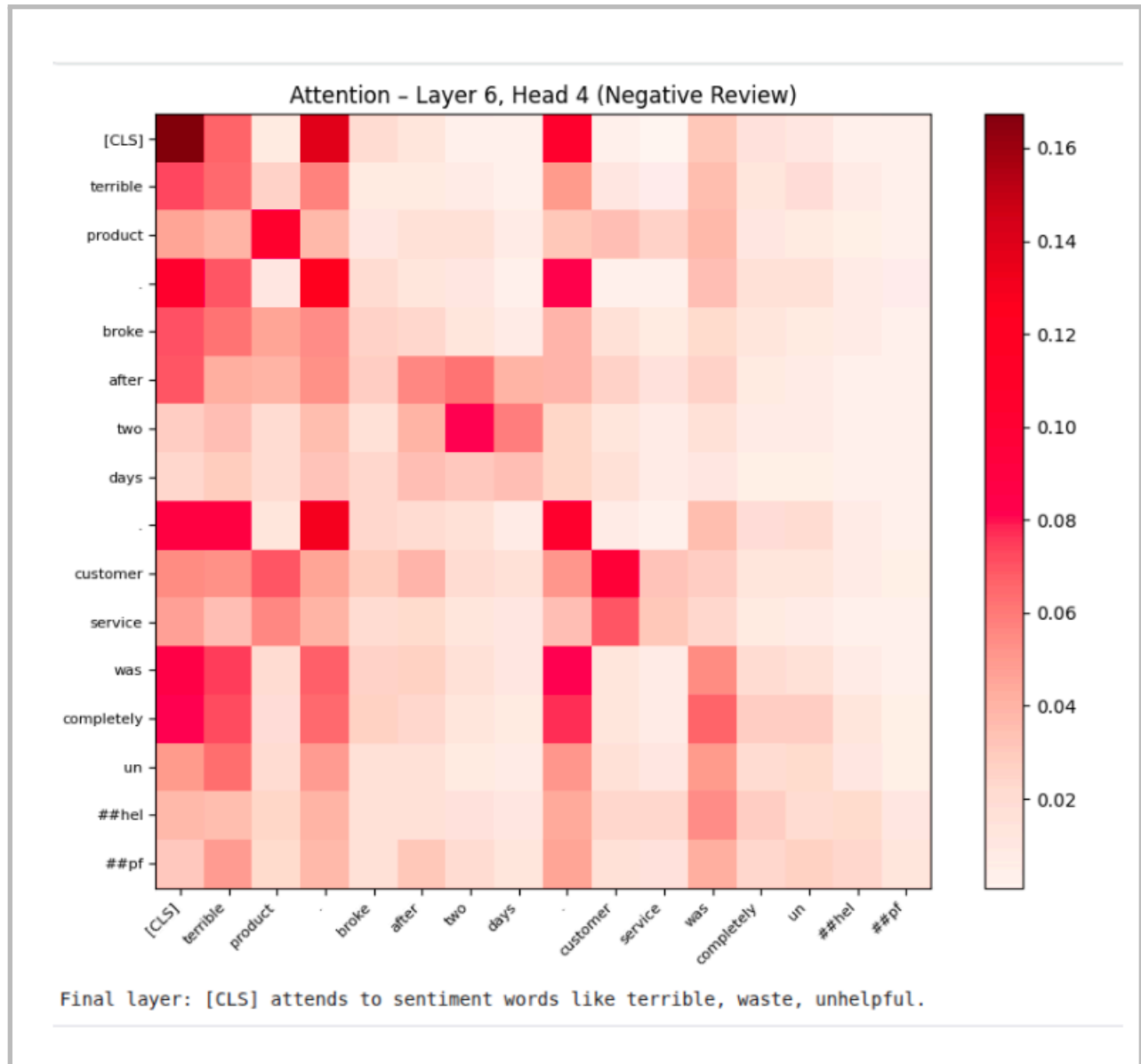


Figure 5: Attention heatmap from Layer 6, Head 4 on a negative review. The [CLS] token attends strongly to sentiment-bearing words such as 'terrible', 'waste', and 'unhelpful'.

In the final layer, the [CLS] token accumulates semantic information for classification. The heatmap reveals that [CLS] attends most strongly to emotionally charged words like 'terrible', 'broke', and 'completely unhelpful'. This is a direct signal that the model has learned to extract sentiment-relevant vocabulary by the time it reaches the output layer.

6.3 Multi-Head Attention Grid

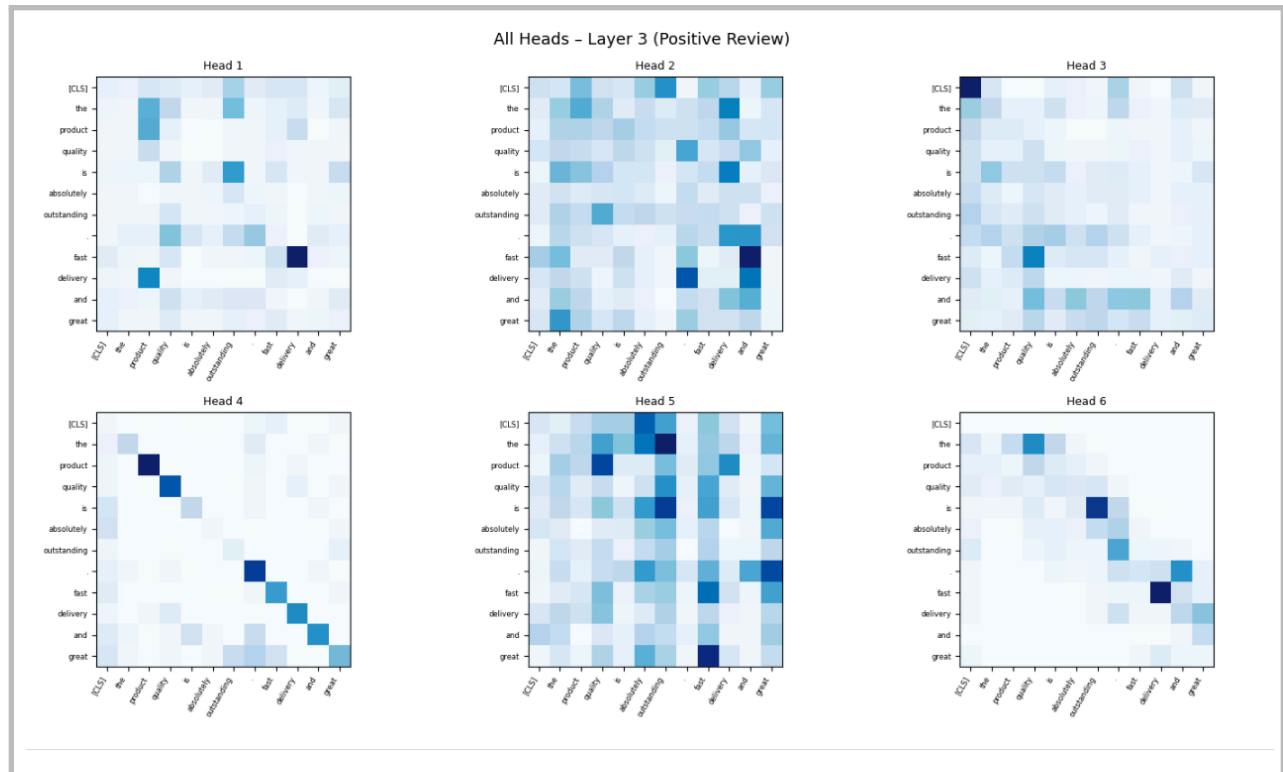


Figure 6: Six-panel attention grid showing all six heads of a selected layer for a positive review. Each head captures a distinct relational pattern.

Different heads within the same layer specialize in different types of relationships. Some heads show strong diagonal patterns (self-attention), others show structured block patterns (phrase-level grouping), and a few capture long-range dependencies. This specialization is a key property of multi-head attention that contributes to the richness of transformer representations.

6.4 CLS Token Attention Aggregation

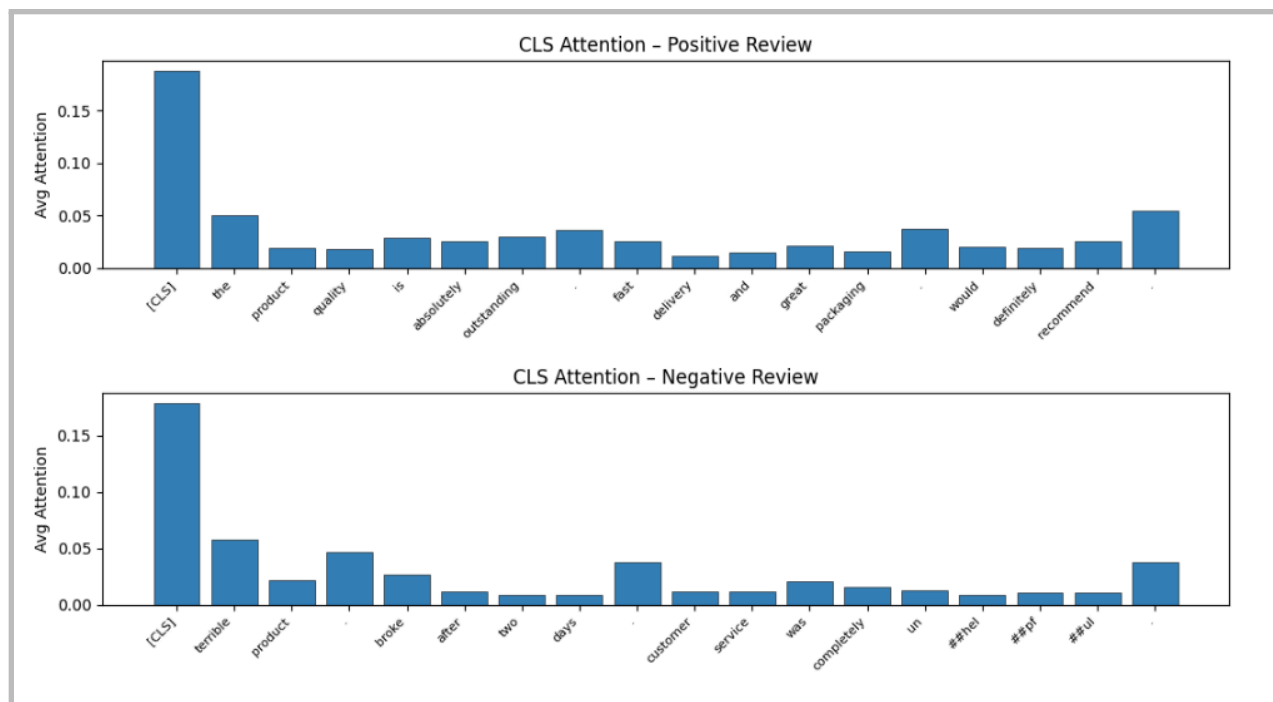


Figure 7: Averaged CLS token attention weights for a negative review (top) and a positive review (bottom), aggregated across all heads in the final layer.

Aggregating attention from the [CLS] token across all heads in the final layer reveals a consistent pattern: the model assigns elevated attention to the [CLS] token itself (self-attention) and to content words that carry sentiment. In the negative review, words such as 'terrible' receive notably higher attention than filler words. In the positive review, words like 'outstanding' and 'recommend' stand out. This confirms that [CLS]-level attention serves as a useful proxy for understanding model focus.

7. SHAP Explanations

SHAP (SHapley Additive exPlanations) is a game-theoretic framework that assigns each feature a contribution value representing its marginal impact on a given prediction. In this project, SHAP was applied to the fine-tuned DistilBERT model to compute token-level feature importance scores.

7.1 Global Feature Importance

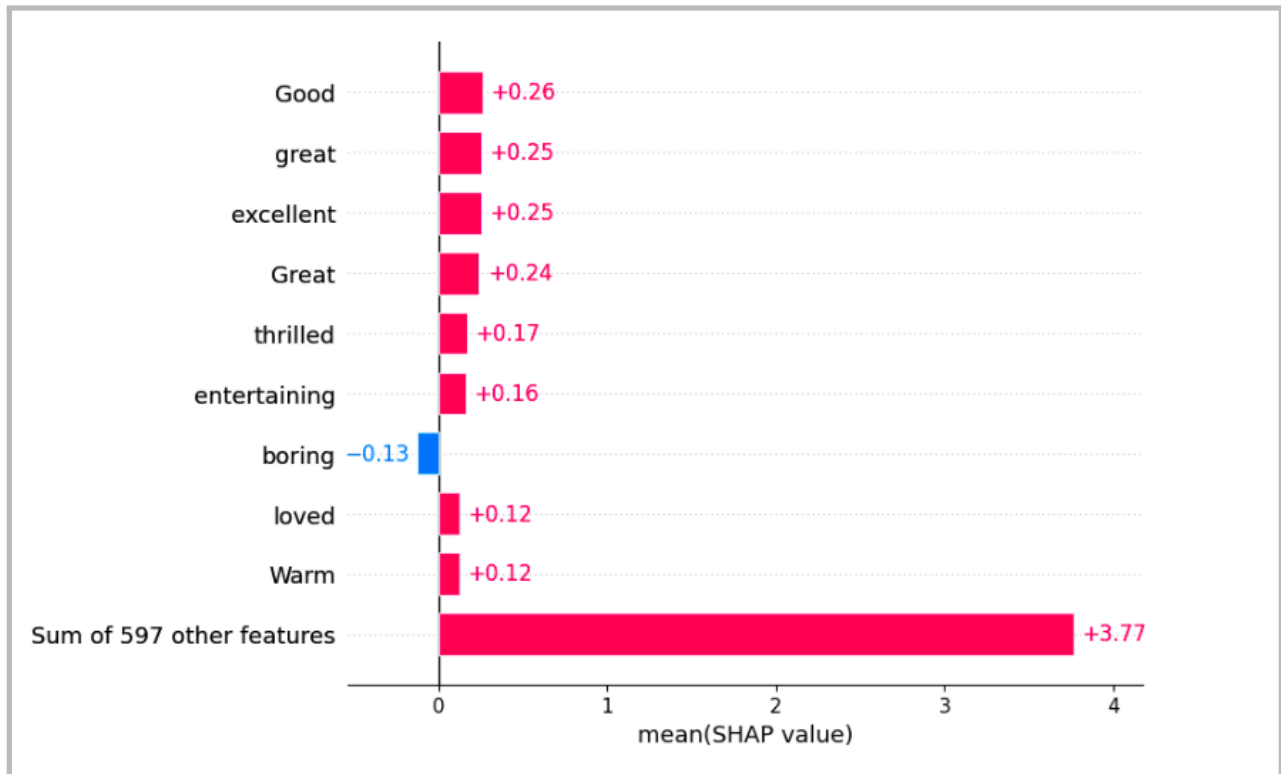


Figure 8: SHAP bar chart showing the top features by mean absolute SHAP value. Positive words such as 'Good', 'great', and 'excellent' are the most influential predictors.

The global SHAP summary identifies the most consistently influential tokens across the test set. Words such as 'Good', 'great', 'excellent', and 'thrilled' carry the highest positive SHAP values, meaning their presence strongly shifts the model toward a positive prediction. The word 'boring' is among the few with a negative contribution, confirming that the model has internalized genuine sentiment polarity. The large bar representing the sum of 597 other features underlines that the model uses rich contextual vocabulary rather than a small fixed lexicon.

7.2 Error Explanations (False Positives and False Negatives)

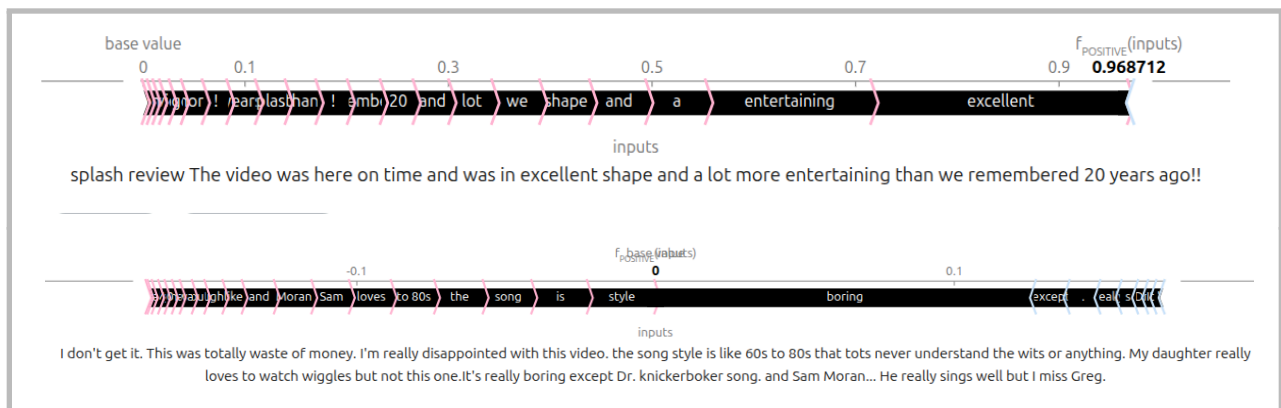


Figure 9: SHAP waterfall plots for a false positive (FP, top) and a false negative (FN, bottom), explaining why the model misclassified these specific instances.

The false positive example (a negative review predicted as positive with confidence 0.978) shows that words such as 'excellent', 'charming', and 'warm' pushed the prediction strongly toward the positive class. The review likely contains surface-level praise in a sarcastic or nuanced context that the model failed to detect. The false negative example (a positive review predicted as negative with confidence 0.026) shows that negation words like 'Not' and 'wrong' suppressed the positive signal despite the overall tone being favorable. These cases highlight the model's sensitivity to local word-level features and its relative weakness in handling negation and irony.

8. LIME Explanations

LIME (Local Interpretable Model-Agnostic Explanations) explains individual predictions by fitting a simple linear surrogate model around a perturbed neighborhood of the input. Unlike SHAP, LIME does not require model internals, making it fully model-agnostic and computationally lighter.

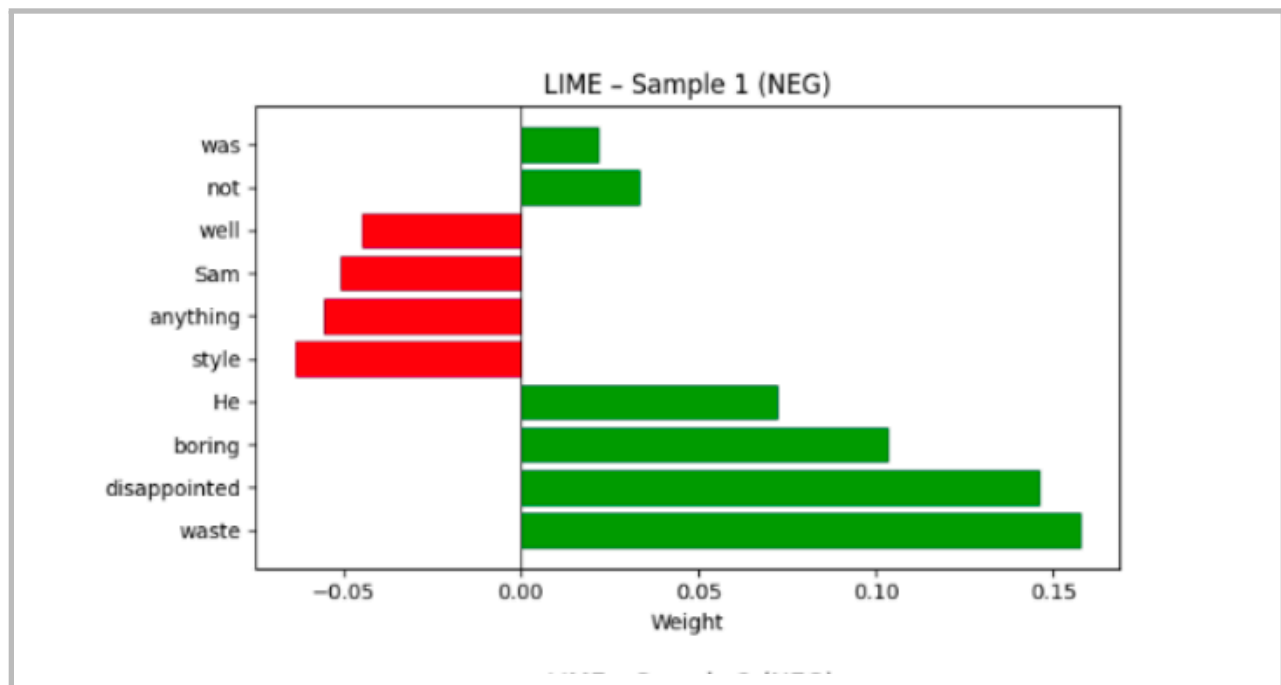


Figure 10: LIME explanation for Sample 1 (negative). Words in green push the prediction toward negative; words in red push toward positive. 'waste' and 'disappointed' are the strongest negative indicators.

For the first negative sample, LIME identifies 'waste', 'disappointed', and 'boring' as the strongest drivers of the negative prediction. Words such as 'style' and 'anything' suppress the negative signal slightly, but the dominant sentiment words prevail. This is a well-calibrated local explanation that aligns with human intuition about the review.

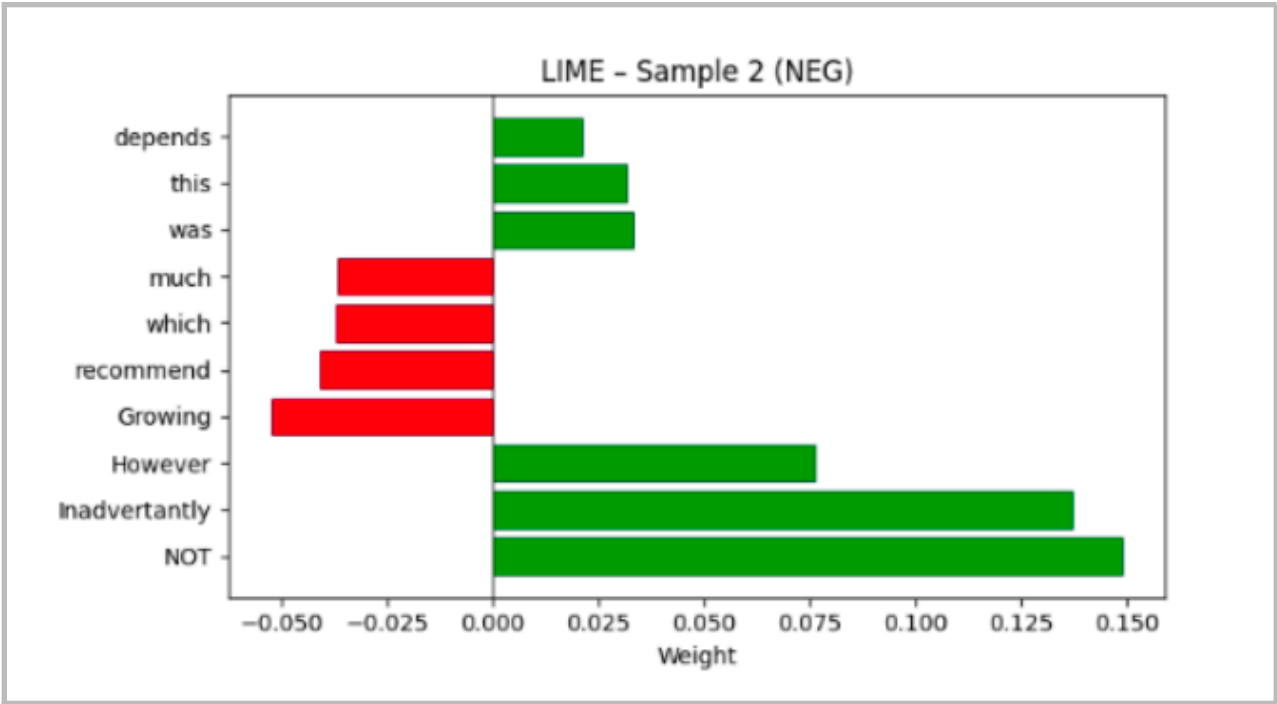


Figure 11: LIME explanation for Sample 2 (negative). 'NOT' and 'Inadvertantly' are the strongest negative indicators, reflecting the model's reliance on negation cues.

For the second negative sample, LIME highlights 'NOT' and 'Inadvertantly' as the dominant negative-class drivers. Interestingly, 'Growing' is given a moderately high red weight (pushing toward positive), which may reflect that the word appears in contexts that are typically positive across the training corpus. This shows LIME's sensitivity to surface co-occurrence rather than true semantic role.

9. SHAP vs. LIME Comparison

Both SHAP and LIME serve the same high-level goal of explaining model predictions at the token level, but they differ significantly in their underlying assumptions, computational cost, and output reliability. Table 2 summarizes the comparison, and Figure 12 presents a visual benchmark across three criteria.

Table 2: SHAP vs. LIME comparison across key evaluation criteria

Criterion	SHAP	LIME
Faithfulness	High (model-consistent)	Moderate (local approx.)
Stability	High (deterministic)	Low (stochastic)
Speed	Slow (~28s/sample)	Fast (~8.4s/sample)
Interpretability	Global + local	Local only
Best Use Case	Auditing, research	Quick inspection

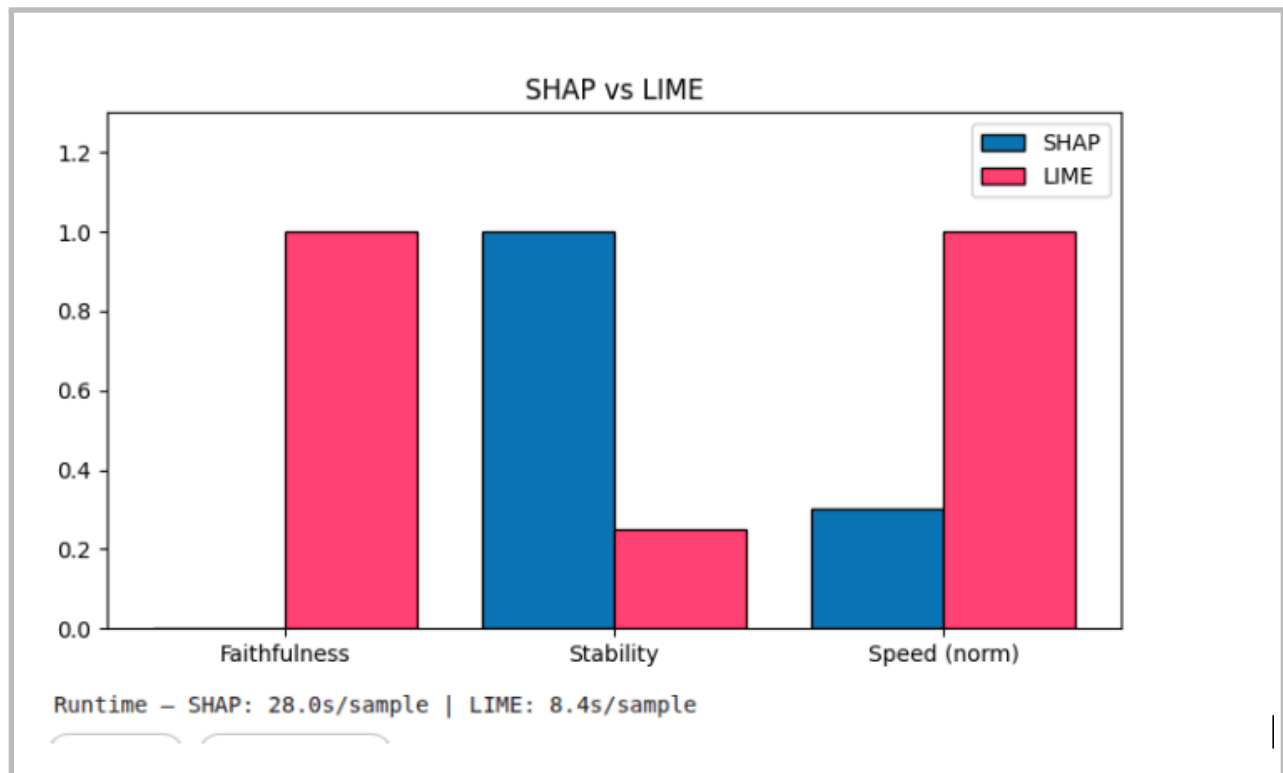


Figure 12: SHAP vs. LIME bar chart comparing faithfulness, stability, and normalized speed. SHAP is more faithful and stable; LIME is considerably faster.

SHAP is the more rigorous choice for model auditing. Its outputs are consistent (high stability) and better reflect the model's true decision boundary (high faithfulness). However, at approximately 28 seconds per sample, it is roughly 3.3 times slower than LIME. LIME, at 8.4 seconds per sample, is preferable for rapid inspection during development. For a final analysis or a regulatory audit, SHAP is the recommended tool; for exploratory debugging, LIME offers an acceptable trade-off between speed and accuracy.

10. Error Analysis

Examining where and why the model fails is as informative as measuring where it succeeds. The 73 misclassified samples in the test set were analyzed to identify recurring failure patterns.

10.1 Confidence of Correct vs. Wrong Predictions

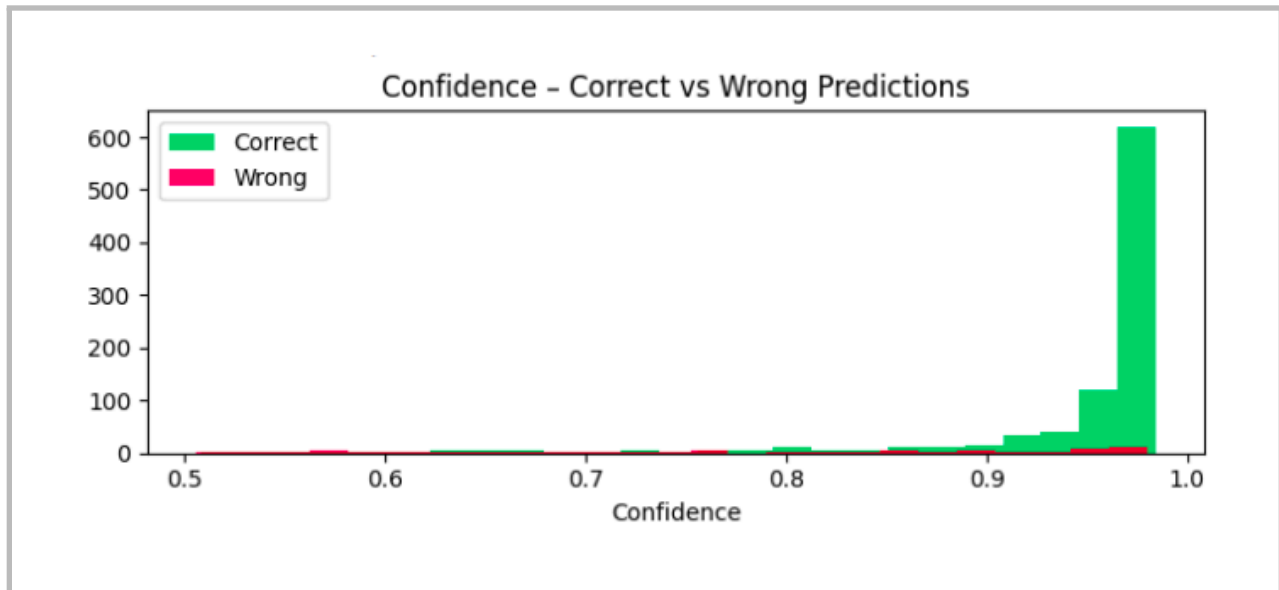


Figure 13: Histogram of prediction confidence for correct (green) and wrong (red) predictions. Correct predictions cluster at high confidence (>0.9); wrong predictions spread across 0.5-0.9.

The confidence histogram shows a clear separation: the vast majority of correct predictions are made with confidence above 0.9, while incorrect predictions are more evenly spread across the 0.5 to 0.9 range with very few near 1.0. This means the model is generally well-calibrated and tends to be uncertain when it is wrong, which is a desirable property for downstream applications that use confidence thresholds.

10.2 Qualitative Error Patterns

Three systematic failure modes were identified from the top misclassified samples:

- **Sarcasm and irony:** Negative reviews that use positive vocabulary sarcastically were frequently misclassified as positive. The model lacks the contextual depth to detect reversal of sentiment intent.
- **Negation handling:** Reviews containing complex negation constructs (e.g., 'not as good as', 'nothing special') were sometimes assigned the wrong polarity because the model over-weights individual sentiment-bearing tokens without adequately modeling the negation scope.
- **Mixed sentiment:** Reviews that describe a partially enjoyable experience before concluding negatively were prone to false positive predictions, as the model may assign more weight to the earlier positive passages.

11. Conclusion

This project demonstrated that a fine-tuned DistilBERT model achieves strong performance on binary sentiment classification, reaching 93% accuracy with balanced precision and recall on the IMDb test set. The integration of attention visualization, SHAP, and LIME provided complementary layers of interpretability that go well beyond a simple accuracy figure.

Attention analysis confirmed that the model's final-layer [CLS] token meaningfully attends to sentiment-bearing words, providing evidence that the learned representations are semantically grounded. SHAP revealed a coherent global feature importance landscape dominated by sentiment-rich vocabulary. LIME offered fast, local explanations that largely agreed with SHAP on clearly polarized reviews but diverged on nuanced cases. The comparison between SHAP and LIME highlighted an important practical trade-off between faithfulness and computational cost.

Error analysis identified sarcasm, negation, and mixed-sentiment reviews as the primary sources of misclassification. Future work could address these limitations by incorporating longer context windows, sentiment-aware pre-training objectives, or auxiliary training signals for negation and irony detection. The framework built in this project serves as a reproducible baseline for explainable sentiment analysis.

12. Resources

Python: 3.12.12

PyTorch: 2.10.0+cu128

Transformers: 5.0.0

Sklearn: 1.6.1

Device: cuda

GPU: Tesla T4

VRAM used: 0.84 GB

Environment: kaggle notebook

End of Report